



Proyecto 1

Tecnológico de Costa Rica
Escuela de Ingeniería en Computación
Redes(IC 7602)
Segundo Semestre 2026

1. Objetivo General

- Implementar servicios de capa de aplicación sobre los protocolos de transporte UDP y TCP.
-

2. Objetivos Específicos

- Desarrollar servidores UDP y TCP en los lenguajes de programación GNU C, Rust y Java, respectivamente.
 - Implementar el procesamiento de algunas peticiones DNS especificadas en el [RFC2929](#).
 - Desarrollar una aplicación que transporte un servicio no orientado a conexión sobre un servicio orientado a conexión.
 - Interactuar con documentación formal de redes.
 - Implementar una API REST en Java.
 - Automatizar una solución mediante el uso de Docker, Helm Charts, Terraform, Docker Compose, Kubernetes o proveedores de nube.
 - Implementar health checks para protocolos TCP y HTTP.
-

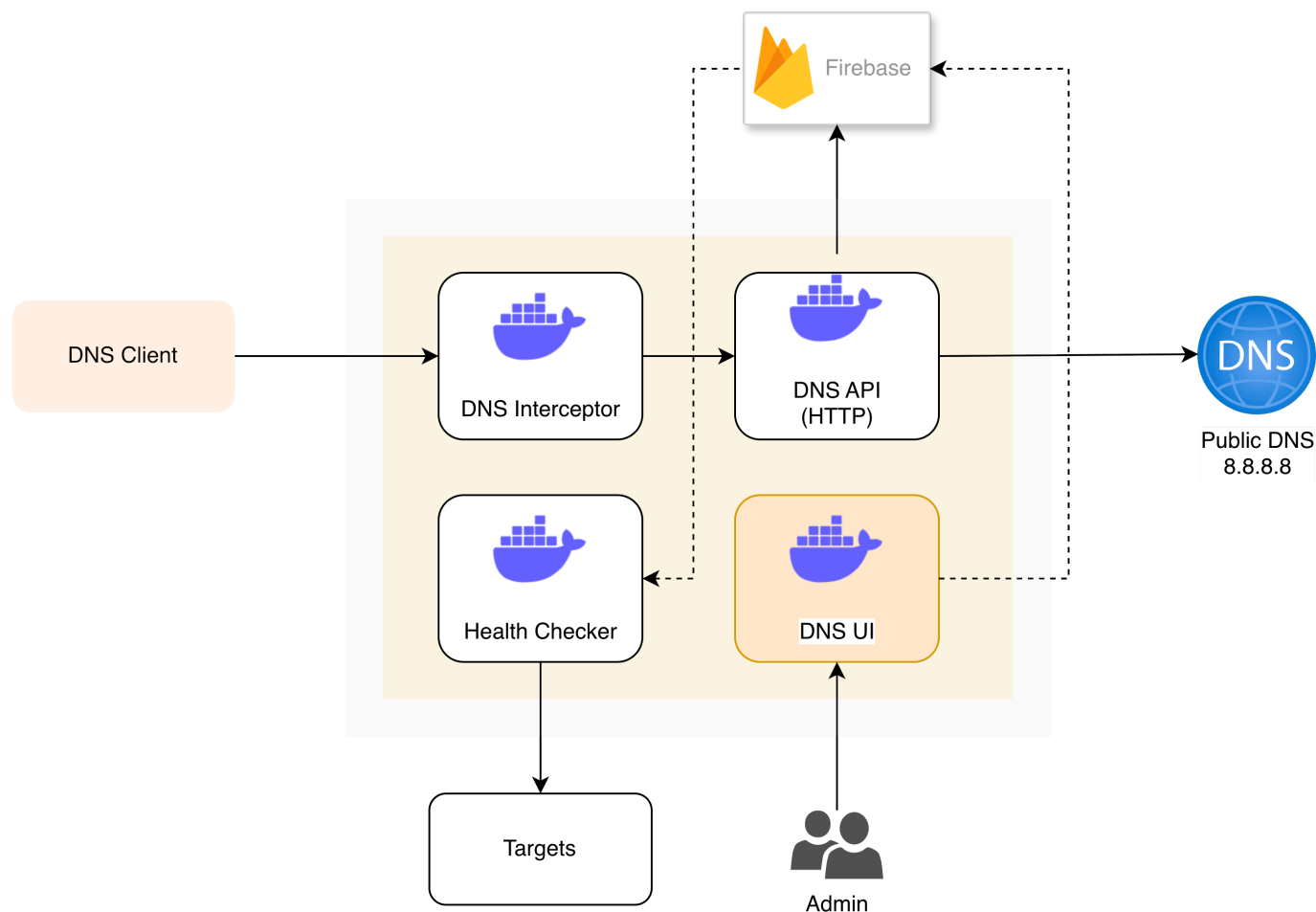
3. Datos Generales

- El valor del proyecto: 20%
- El proyecto debe ser implementada en grupos de máximo 5 personas.
- Cualquier indicio de copia será calificado con una nota de 0 y será procesado de acuerdo con el reglamento. La copia incluye código/configuraciones que se puede encontrar en Internet y que sea utilizado parcial o totalmente sin el debido reconocimiento al autor.
- La revisión se realiza de forma presencial mediante una evaluación escrita, para cuya resolución deben estar presentes todos los integrantes del grupo.
- Se debe incluir documentación con instrucciones claras para ejecutar el proyecto.

- Se espera que todos y todas las integrantes del grupo entiendan la implementación suministrada.
 - Se deben seguir buenas prácticas de programación en el caso de que apliquen. Por ejemplo, documentación interna y externa, estándares de código, diagramas de arquitectura, diagramas de flujo, pruebas unitarias son algunas de las buenas prácticas que se esperan de un estudiante de Ingeniería en Computación.
 - El lenguaje de implementación, en caso de ser requerido y no especificado por el profesor, deberá ser seleccionado por cada grupo y debe quedar debidamente documentado. No se permite el uso de Python.
 - Toda documentación debe ser implementada en Markdown.
 - Si la instalación de servicios no se encuentra automatizado se obtendrá una nota de 0. Si el proyecto no se entrega completo, la porción que sea entregada debe estar completamente automatizada, de lo contrario se obtendrá una nota de 0.
 - En caso de no entregar documentación se obtendrá una nota de 0.
 - El email de entrega debe contener una copia del proyecto en formato tar.gz y un link al repositorio dónde se encuentra almacenado, debe seguir los lineamientos en el documento [Lineamientos para el envío de emails.pdf](#).
 - Los grupos de trabajo deben autogestionarse; la persona facilitadora del curso no es responsable si un miembro del equipo no realiza su trabajo.
 - En consultas presenciales, por correo electrónico o por Telegram, como mínimo se debe haber leído sobre el tema y haber intentado comprenderlo. Las consultas deben ser concretas y se debe demostrar que se intentó resolver el problema en cuestión.
 - El uso de Inteligencia Artificial (IA) estará regulado por los lineamientos establecidos en el documento [Política para el Uso de Inteligencia Artificial Generativa.pdf](#).
-

4. Descripción

El Proyecto I busca profundizar en los temas de la capa de aplicación y la capa de transporte mediante la implementación de aplicaciones que utilicen los protocolos TCP y UDP. Como parte de este ejercicio, se deberá implementar un servicio de DNS con algunas de las características definidas en el [RFC1035](#).



Se deberán crear los siguientes componentes:

DNS Interceptor

Este es un contenedor Docker que ejecuta una pequeña aplicación escrita en lenguaje de programación Rust. La misma escuchará en el puerto UDP/53. Esta aplicación recibirá paquetes del protocolo DNS y, al recibirlos, deberá examinarlos siguiendo la especificación oficial del [RFC2929](#) y tomar alguna de las siguientes acciones:

1. En el caso de recibir un paquete diferente a un query estándar ($QR = 0$ y $OPCODE = 0$), deberá codificar la solicitud en BASE64 y enviarla vía HTTPS (POST `/api/dns_resolver`) al componente llamado DNS API. Cuando se reciba la respuesta, esta vendrá en BASE64; la misma debe ser decodificada y enviada al cliente solicitante.
2. En el caso de recibir un paquete de tipo query estándar ($QR = 0$ y $OPCODE = 0$), deberá examinarlo detenidamente e identificar el host que se está tratando de resolver. Una vez identificado, deberá consultar en Supabase/Firebase, a través del DNS API, si existe un registro para este host y tomar alguna de las siguientes acciones:
 - Si existe, deberá extraer la información de esta base de datos y retornarla. La información devuelta debe cumplir al 100% con la especificación de los protocolos definidos en los RFC 2929 y RFC 1035, para asegurar que clientes como nslookup y los sistemas de resolución de nombres de Windows/Linux puedan comunicarse con el componente sin inconvenientes. Se pueden presentar los siguientes casos al obtener el valor desde Supabase/Firebase:
 - El tipo de registro es "single": en este caso, se tiene una única IP y se retorna dicha IP.

- El tipo de registro es "multi": se tienen múltiples IPs; se espera que el estudiantado implemente una estrategia de round-robin, de modo que diferentes peticiones retornen IPs distintas.
 - El tipo de registro es "round-trip": se deberá retornar la IP con menor latencia según la cercanía con la persona usuaria.
 - El tipo de registro es "weight": se tienen múltiples IPs con un peso asociado; la IP con mayor peso deberá atender más peticiones.
 - El tipo de registro es "geo": se tienen IPs asignadas por país; se toma la source IP de la petición original y se consulta la base de datos [IP to Country](#) (la cual debe estar cargada en Supabase/Firebase) para identificar el país correspondiente. Con base en esto, se retorna la IP adecuada. En caso de no existir un registro para el país, se retornará una IP de forma aleatoria.
- En caso de no existir el registro o que la IP asignada se encuentre unhealthy, la solicitud se tratará como el primer caso.

Nota: Esta aplicación deberá permitir múltiples solicitudes de forma simultánea.

DNS API

Un contenedor que ejecuta una API REST implementada en Java, la cual expone los métodos necesarios para el funcionamiento de la aplicación. Como mínimo, se requieren dos métodos:

- `/api/exists`: Este método se utilizará para verificar si un dominio existe en Supabase/Firebase.
- `/api/dns_resolver`: Este método recibe en el cuerpo (data) un paquete DNS (RFC 2929) codificado en BASE64. Dicho paquete deberá ser decodificado y enviado a un servidor DNS remoto para su resolución. Esto implica lo siguiente:
 - La aplicación debe implementar una API REST (pueden utilizar cualquier framework). También deberá implementar un cliente UDP/DNS para enviar las solicitudes al servidor DNS remoto.
 - Debe recibir como parámetro o archivo de configuración la IP de un servidor DNS remoto.
 - Debe soportar múltiples peticiones concurrentes.

Cualquier otro método requerido en la API deberá ser definido por cada grupo. Toda la información nativa de DNS (paquetes) intercambiada entre el DNS Interceptor y el DNS API debe codificarse en BASE64.

Health Checker

Este es un contenedor Docker que ejecuta una pequeña aplicación escrita en GNU C. Esta aplicación lee los registros almacenados en Supabase/Firebase y realiza los health checks correspondientes, marcando cada registro como healthy o unhealthy (mayoría simple).

La aplicación debe ser capaz de monitorear servicios con basic auth. Cada Health Checker tiene asociada una ubicación geográfica (simulada para efectos del proyecto) y medirá el round-trip time de las pruebas; esta información será utilizada para implementar el tipo de registro round-trip.

DNS UI

Un contenedor que ejecuta una aplicación en React que permite:

- Crear, borrar y modificar registros de tipo "single", "multi", "weight", "round-trip" y "geo", junto con la definición de sus respectivos health checks.

Tipos de health checks:

- TCP: Verifica si una conexión puede establecerse; se deben especificar timeout, retries y el intervalo entre pruebas (por ejemplo, cada 30 segundos).
- HTTP: Se especifica un path, timeout, retries, intervalo entre pruebas (por ejemplo, cada 30 segundos) y los códigos HTTP esperados en la respuesta. Crear, borrar y modificar registros de la base de datos [IP to Country](#).

Para este apartado se permite el uso de IA generativa; sin embargo, se deben documentar tanto los prompts utilizados como los problemas encontrados durante el proceso.

El proyecto debe ejecutarse en Docker. Puede correr localmente en Kubernetes o en una máquina virtual dentro de algún proveedor de nube.

Registro round-trip y Health Checker

Para una implementación satisfactoria de los registros de tipo round-trip, es necesario contar con múltiples Health Checkers en diferentes ubicaciones geográficas. Cada uno deberá tener asociada información de latitud, longitud, país y ciudad.

Cuando un Health Checker realiza una prueba, envía la información a la base de datos Supabase/Firebase. Cuando un cliente solicita la resolución de un nombre de tipo round-trip, se puede determinar su país y ciudad mediante la base de datos [IP to Country](#) (algunas versiones incluyen la ciudad; en caso contrario, se trabajará únicamente con el país).

Con esta información, es posible estimar la ubicación del cliente (latitud, longitud, país y ciudad) y compararla con la de los distintos Health Checkers para seleccionar el más cercano y con menor latencia.

Debido a la complejidad de implementar este tipo de pruebas, se invita a los grupos a realizar alguna simulación o aplicar penalizaciones en los round-trip para verificar que su implementación funciona correctamente.

Automatización

Todo el trabajo que se implemente debe estar automatizado mediante las siguientes herramientas:

- Kubernetes/Docker.
- Helm Charts/Docker Compose.

No se deben tener valores quemados a nivel de código; cualquier configuración debe ser inyectada mediante variables de entorno.

Para este apartado se permite el uso de IA generativa; sin embargo, se deben documentar tanto los prompts utilizados como los problemas encontrados durante el proceso.

5. Documentación

Se deberá entregar una documentación que al menos debe incluir:

- Instrucciones claras de como ejecutar su proyecto.
- Pruebas realizadas, con pasos para reproducirlas.
- Recomendaciones y conclusiones (al menos 10 de cada una).
- La documentación debe cubrir todos los componentes implementados o instalados/configurados, en caso de que algún componente no se encuentre implementado, no se podrá documentar y tendrá un impacto en la completitud de la documentación.
- Se debe incluir una tabla en la que se resuma qué funciona y qué no funciona en la implementación suministrada, para cada uno de los siguientes aspectos:
 - DNS Interceptor
 - Tipo de registro es "single"
 - Tipo de registro es "multi"
 - Tipo de registro es "round-trip"
 - Tipo de registro es "weight"
 - Tipo de registro es "geo"
 - DNS API
 - Health Checker
 - DNS UI

Imaginen que la documentación está siendo creada para una persona con conocimientos básicos en el área de computación y ustedes esperan que esta persona pueda correr su proyecto y probarlo sin ningún problema, el uso de imágenes, diagramas, code snippets, videos, etc. son recursos que pueden ser útiles.

6. Recomendaciones

- Utilizar nslookup para interactuar con el DNS Resolver.
 - Utilizar postman para interactuar con el DNS API
 - Si su proyecto funciona correctamente, podrían modificar el archivo resolv.conf de sus computadoras, ponerlo a apuntar al IP de su contenedor y estas deberían seguir funcionando sin problemas.
 - Realicen peer-review/checkpoints semanales y no esperen hasta el último momento para integrar su aplicación.
-

7. Uso de GitHub y buenas prácticas de desarrollo de software

Cada grupo deberá aplicar los lineamientos especificados en el documento [Uso de GitHub.pdf](#). El incumplimiento de estos requerimientos tendrá un impacto en la nota final del proyecto.

A partir de la fecha de asignación del proyecto, cada grupo deberá entregar **todos los jueves** un reporte semanal de avance.

El reporte deberá incluir:

- La lista de tareas que se encuentran en desarrollo.
- La persona o personas asignadas a cada tarea.
- El estado y avance de cada tarea.
- Los problemas encontrados durante la semana.
- Una lista de los **commits** realizados por cada integrante, incluyendo su descripción y los archivos modificados.
- Una lista de los **Pull Requests (PRs)** creados por cada integrante, incluyendo su descripción y los cambios realizados.

Para el **primer reporte**, únicamente se deberá presentar la lista de tareas planificadas, una breve descripción de cada una y la persona o personas responsables de realizarlas.

A partir del **segundo reporte**, se deberá presentar el avance de cada tarea, los problemas encontrados y la evidencia del trabajo realizado mediante los commits y Pull Requests correspondientes.

El reporte deberá reflejar claramente la participación y el progreso individual de cada integrante del grupo.

Se deberá seguir el formato establecido en el documento [Reporte Avance.md](#).

En caso de que se extienda la fecha de entrega, se se deben entregar de avance adicionales.

El asunto (subject) del correo electrónico deberá utilizar el siguiente formato: **P1R{NÚMERO DE REPORTE}**. Por ejemplo, para el segundo reporte de avance, el asunto deberá ser **P1R2**.

8. Entregables

→ Documentación.

→ Docker files, Docker Compose files y Helm Charts junto con todos los archivos/scripts requeridos para ejecutar su proyecto.

→ Se deberá entregar un video de no más de 10 minutos en el que se demuestre el proyecto implementado. El video deberá incluir una demostración de las funcionalidades desarrolladas, así como una explicación clara del código implementado.

9. Evaluación

Rubro	Valor
Documentación	10%
DNS Interceptor	10%
DNS API	5%
DNS UI	5%
Health Checker	5%
Reportes de avance y uso de GitHub	5%
Evaluación presencial	60%

(*) Todo tiene que estar debidamente automatizado, de lo contrario se calificará con una nota de 0.

10. Fechas importantes

→ **Entrega:** 22/09/26 antes de media noche.

→ **Evaluación presencial:** 29/09/26 durante la clase.

→ **Reportes de Avance:** Cada jueves, antes de la medianoche, a partir de la fecha de asignación de este proyecto.

11. Evaluación presencial

La evaluación presencial consiste en responder, de manera grupal, una serie de preguntas relacionadas con la implementación y la solución entregada. Su objetivo es demostrar el conocimiento, la comprensión y la autoría de la solución desarrollada por las personas estudiantes.

La evaluación tendrá una duración de 90 minutos, es presencial y deberá realizarse con los mismos grupos de trabajo conformados para el desarrollo del proyecto.